



DD2350 ALGORITMER, DATASTRUKTURER OCH KOMPLEXITET

Övning 7

Probabilistiska algoritmer och reduktioner

Marcus | HT 2026

Uppgifter och lösningsförslag: **Viggo Kann**.

Bygger på tidigare övningsbilder av **Anton Grensjö** (2017), **Lisa Li** (2019) och **Alice** (2020).

Sandfärgade bilder med kanin 🐇 är kaninhål: fördjupningar som går att hoppa över.

Den här övningen står på andras axlar

Viggo Kann

Uppgifterna och lösningsförslagen, bland annat algoritmen CREATECOLOURING.

Lisa Li (övningsbilder 2019)

Diagonalexemplet för Freivalds algoritm. Pseudokoden för kantsammanhanget.

Anton Grensjö (övningsbilder 2017)

Parningsargumentet för Freivalds algoritm. Uppställningen $f(n) = g(n + 20)$ i medianuppgiften.

Alice (övningsbilder 2020)

LV-väskan. RandomLevel steg för steg. s/t -exemplet. Färgningen av C_5 .

Idéerna ovan är deras. Motorcykeln och BOK/BIO är mina minnesregler — liksom kaninhålen, motexemplen och felen som eventuellt har smugit sig in.

Dagens program

Probabilistiska algoritmer

1. Skipplistor
2. Verifikation av matrismultiplikation

varför inte $p = 1/e$?

Freivalds

Reduktioner

3. Negativt resultat
4. Positivt resultat
5. Optimering som beslut
6. Beslut, värde, konstruktion

medianen

kantsammanhang

jobbmaskning

graffärgning

Tre sorters bilder

 Lösningar och exempel

  Kaninhål (valfria)

  Vanliga missuppfattningar

Temana möts på vägen: uppgift 4 löses med en reduktion till maxflöde — och, i ett kaninhål, med en Monte Carlo-algoritm.

Las Vegas: alltid rätt svar



tiden: slump



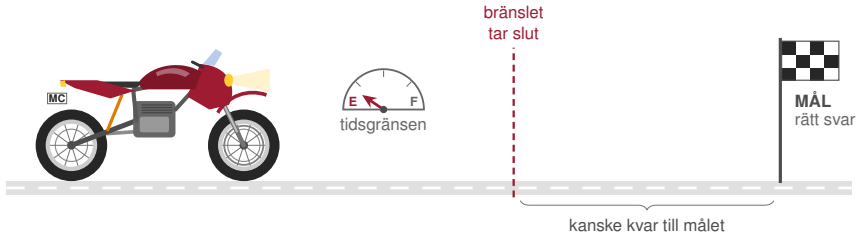
svaret: rätt

- ▶ Svaret är **alltid rätt**. Det är *tiden* som är slumpmässig: vi kan bara lova något om den förväntade tiden.
- ▶ Exempel: quicksort med slumpat pivotelement, skipplistor (uppgift 1).

Minnesregel: en LV-väska är alltid rätt, oavsett tidpunkt. (Alice)

⚠ Missuppfattning: "Probabilistisk betyder att svaret kan bli fel." Inte för Las Vegas. Där är det bara tiden som beror på slumpen.

Monte Carlo: alltid i tid



- ▶ Har en **tidsgräns**. Svaret kan bli fel eller bara ungefärligt — med en risk eller ett fel som går att begränsa.
- ▶ Exempel: **strålföljning** (ray tracing) med slumpade strålar ger bilden i tid, men med brus. Också Freivalds (uppgift 2) och Karger (uppgift 4).

Minnesregel: en MC kör så långt bränslet räcker — och når kanske inte målet. (Marcus)

Exempel: strålföljning är Monte Carlo



- ▶ Varje pixel räknas ut genom att följa **slumpade ljusvägar** genom scenen och ta medelvärdet.
- ▶ Algoritmen slutar när tiden är slut, inte när bilden är perfekt.
- ▶ Svaret blir **ungefärligt**: det vita gruset på den svarta kåpan är brus.

Bilden kommer alltid i tid, men är bara nästan rätt. En brusig MC i Monte Carlo.

Kaninhål: Varför bruset krymper som $1/\sqrt{N}$

En pixel i halvskugga: hur stor del q av lampan syns därifrån? Skicka N slumpade skuggstrålar mot lampan och räkna andelen som kommer fram.



$N = 1$



$N = 4$



$N = 16$



$N = 64$



exakt

- Medelvärdet blir rätt i väntevärde. Bruset — standardavvikelsen — är $\sqrt{q(1-q)/N}$.
- Fyra gånger fler strålar halverar bruset. Tidsgränsen är N , felet är bruset.

⚠ Missuppfattning: "Strålföljning är alltid slumpmässig." Klassisk strålföljning (Whitted, 1980) är deterministisk. Det är de slumpade stickproven — mjuka skuggor, oskärpa, indirekt ljus — som gör den till Monte Carlo (Cook, Porter och Carpenter 1984; Kajiya 1986).

Kaninhål: Från Las Vegas till Monte Carlo — och tillbaka

LV $\Rightarrow$ MC: avbryt. Låt förväntad tid vara $E[T]$.
 Avbryt efter $2E[T]$ steg och svara på måfå.
 Markovs olikhet:

$$P(T > 2E[T]) \leq \frac{1}{2}.$$

Nu har algoritmen en tidsgräns — och en felrisk.

MC $\Rightarrow$ LV: kontrollera. Om svaret går att kontrollera billigt: kör om tills kontrollen godkänner. Alltid rätt svar, slumpmässig tid.
 Faktorisering passar: en faktor kontrolleras med en division.

Freivalds algoritm går inte att göra om till Las Vegas på det sättet. Svaret " $AB = C$ " kan inte kontrolleras billigt — då hade vi inte behövt algoritmen.

1

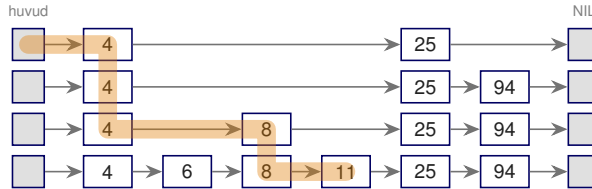
Skipplistor

Målet: förstå varför teorins optimum $p = 1/e$ inte är det man väljer i praktiken.

Att se upp med:

- ▶ "Optimal" betyder optimal *i ett visst kostnadsmått*.
- ▶ Slumpbitar, minne och spridning kostar också.
- ▶ En skipplista svarar alltid rätt. Det är tiden som är slumpmässig.

Skipplistan på en bild



- ▶ Nivå 1 är en vanlig sorterad länkad lista.
- ▶ Ett element på nivå i finns också på nivå $i + 1$ med sannolikhet p , oberoende av allt annat.
- ▶ Sök från översta nivån: gå höger så länge nästa nyckel är mindre, annars ner.
- ▶ Orange: sökningen efter 11. Med sju element vinner man inget; det lönar sig när n växer.

Uppgift 1: Skipplistor

Sannolikheten p att ett element på nivå i i en skipplista också finns på nivå $i + 1$ kan man variera efter behov. Kortaste förväntade sökstigen fås om $p = 1/e \approx 0,3679$. Varför kan det trots detta vara bättre att välja $p = 1/2$ eller $p = 1/4$?

Antal nivåer: ungefär $\log_{1/p} n$. Förväntat antal steg per nivå: ungefär $1/p$. Alltså

$$\text{sökstig} \approx \frac{\log_{1/p} n}{p} = \frac{\ln n}{p \ln(1/p)}.$$

Minimera $\frac{1}{p \ln(1/p)}$: maximera $p \ln(1/p)$.

Derivatan är $\ln(1/p) - 1$, som är noll för $p = 1/e$.

Så långt teorin. Frågan är vad den *inte* räknar med.

Svaret: slumpbitar ska vara billiga

RandomLevel, godtyckligt p

- 1: $nivå \leftarrow 1$
- 2: **medan** SLUMPTAL() $< p$ **och** $nivå < MaxNivå$ **gör**
- 3: $nivå \leftarrow nivå + 1$
- 4: **returnera** $nivå$

Ett slumpflyttal per nivå.

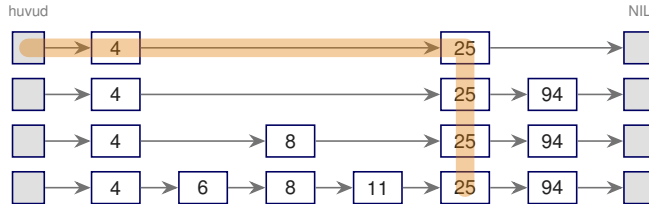
RandomLevel, $p = 2^{-k}$

- 1: $nivå \leftarrow 1$
- 2: **medan** nästa k slumpbitar alla är 1 **och** $nivå < MaxNivå$ **gör**
- 3: $nivå \leftarrow nivå + 1$
- 4: **returnera** $nivå$

k bitar och en jämförelse per nivå.

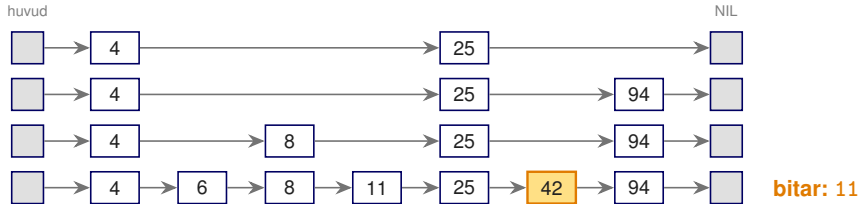
Med $p = 1/2$ räcker ett slumpat maskinord till hela nivån: nivån är 1 + antalet ettor i slutet av ordet. Med $p = 1/e$ behövs flyttal: ett slumptal per nivå, eller en logaritm.

RandomLevel med $p = 1/4$ (Alices exempel)



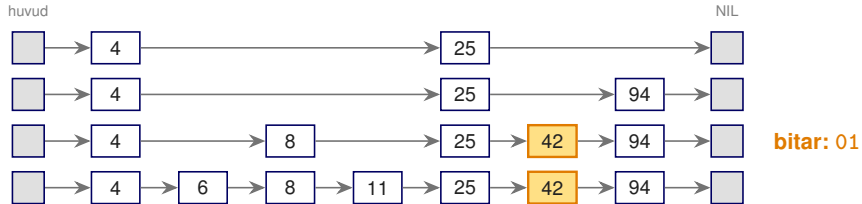
Sök efter 42 som vanligt. Föregångaren är 25 på *varje* nivå — där ska 42 in.

RandomLevel med $p = 1/4$ (Alices exempel)



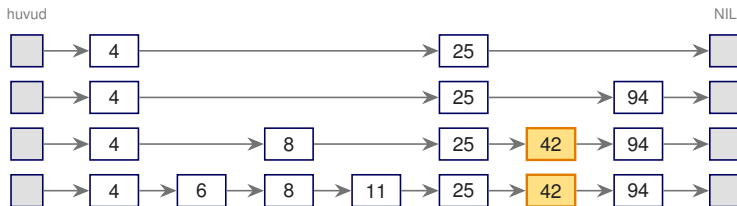
Sätt in 42 på nivå 1. Läs $k = 2$ slumpbitar: 11. Båda ettor — upp en nivå.

RandomLevel med $p = 1/4$ (Alices exempel)



42 finns nu på nivå 2. Nya bitar: 01. Inte båda ettor — stopp.

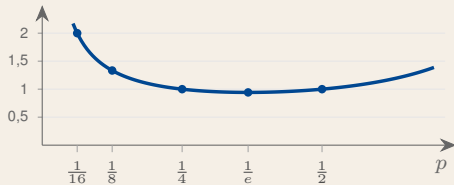
RandomLevel med $p = 1/4$ (Alices exempel)



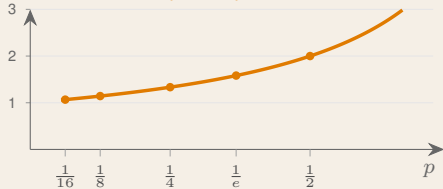
Klart: 42 fick höjd 2. Varje nivå uppåt kostade två bitar, och chansen att gå upp var $(1/2)^2 = 1/4 = p$.

Kaninhål: Hur mycket vinner man egentligen med $1/e$?

förväntad sökstig, relativt $p = \frac{1}{2}$



förväntat antal pekare per element



p	sökstig	pekare
$1/2$	1,00	2,00
$1/e$	0,94	1,58
$1/4$	1,00	1,33
$1/8$	1,33	1,14
$1/16$	2,00	1,07

Axlarna: x är p . Inget är i sekunder. Vänster: förväntat antal steg i sökningen, delat med samma tal för $p = 1/2$ — en kvot. Höger: förväntat antal pekare per element, $1/(1 - p)$.

Minimum är platt: $1/e$ vinner 6% över $1/2$ och $1/4$, men $1/4$ har en tredjedel färre pekare än $1/2$. Pugh, som uppfann skipplistan, rekommenderar $p = 1/4$ — utom när spridningen i söktid är viktig.

Vanliga missuppfattningar om skipplistor

⚠ **Missuppfattning:** " $p = 1/e$ är optimalt, så alla andra val är sämre." Optimalt i ett enda mått, med ett platt minimum. Minne och slumpbitar räknas inte där.

⚠ **Missuppfattning:** "Mindre p ger snabbare sökning." Färre nivåer, men längre steg på varje: med $p = 1/16$ tar sökningen dubbelt så lång tid som med $1/2$.

⚠ **Missuppfattning:** "Slumpen kan ge fel svar." Skipplistan är Las Vegas: svaret är alltid rätt, bara tiden varierar.

⚠ **Missuppfattning:** "Sökning tar $O(\log n)$." I förväntan. I värsta fall $O(n)$, om alla element råkar få samma höjd. Det är bara mycket osannolikt.

2

Verifikation av matrismultiplikation

Målet: kontrollera $AB = C$ i tid $O(kn^2)$ med felrisk högst 2^{-k} — och kunna bevisa felrisken.

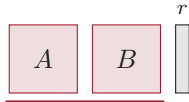
Att se upp med:

- ▶ Ordningen: $A(Br)$ är billigt, $(AB)r$ är dyrt.
- ▶ Felet är ensidigt: svaret "nej" är alltid rätt.
- ▶ Varför minst hälften av alla r avslöjar ett fel.

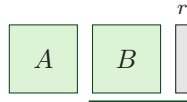
Uppgift 2: Verifikation av matrismultiplikation

Snabbaste kända metoden för multiplikation av två $n \times n$ -matriser är $O(n^{2,376})$. Visa att det går att probabilistiskt verifiera en matrismultiplikation i kvadratisk tid: konstruera en Monte Carlo-algoritm som i tid $O(kn^2)$ verifierar att $AB = C$ och har en felsannolikhet på högst 2^{-k} .

Idé: om $C = AB$ så är $Cr = A(Br)$ för *varje* vektor r . Testa med ett slumpat r .



först AB :
 $O(n^{2,37\dots})$



först Br , sedan $A(Br)$:
 $O(n^2)$

Algoritmen

VerifyMult(n, A, B, C, k)

```

1: för  $i \leftarrow 1$  till  $k$  gör
2:    $r \leftarrow$  slumpvektor i  $\{0, 1\}^n$ 
3:    $a \leftarrow C \cdot r$ 
4:    $b \leftarrow A \cdot (B \cdot r)$ 
5:   om  $a \neq b$  så
6:     returnera falskt
7: returnera sant
  
```

- ▶ k varv, tre produkter matris $\cdot$ vektor per varv, $O(n^2)$ var: $O(kn^2)$.
- ▶ Med $r \in \{0, 1\}^n$ blir det bara additioner.
- ▶ Kvadratisk tid är *linjär* här: indata är tre matriser med $3n^2$ tal.

⚠ Missuppfattning: "Räkna ut AB och jämför med C ." Det kostar $O(n^{2,37\dots})$ med den bästa kända metoden — mer än kvadratiskt.

Exempel: ett enda fel i C

$$AB = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}, \text{ men } C = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & \mathbf{17} \end{pmatrix}.$$

(Exemplet: Lisa Li)

Stickprov 1: $r = (1, 1, 0)^T$

$$Cr = (1, 2, 0)^T, \quad ABr = (1, 2, 0)^T$$

Lika: algoritmen blir lurad.

Stickprov 2: $r = (0, 1, 1)^T$

$$Cr = (0, 2, \mathbf{17})^T, \quad ABr = (0, 2, 3)^T$$

Olika: $AB \neq C$ är bevisat.

Felet sitter i kolumn 3. Det syns precis när $r_3 = 1$ — alltså med sannolikhet exakt $1/2$. Gränsen $1/2$ går inte att förbättra.

Varför avslöjar minst hälften av alla r felet?

- ▶ Låt $D = AB - C \neq 0$, med $d_{ik} \neq 0$.
- ▶ Para ihop varje r med $r' =$ samma vektor med bit k vänd.
- ▶ Då är $(Dr')_i = (Dr)_i \pm d_{ik}$. Båda kan inte vara noll.
- ▶ Minst en vektor i varje par avslöjar felet. Högst hälften lurar algoritmen.

Rad i i D är $(1, -1, 2)$, så $(Dr)_i = r_1 - r_2 + 2r_3$:

$r_3 = 0$	(0, 0, 0) → 0	(1, 0, 0) → 1	(0, 1, 0) → -1	(1, 1, 0) → 0
$r_3 = 1$	(0, 0, 1) → 2	(1, 0, 1) → 3	(0, 1, 1) → 1	(1, 1, 1) → 2

pilarna: vänd r_3

grön: avslöjar röd: lurad. Här lurar 2 av 8.

(Argumentet: Anton Grensjö)

k oberoende varv: felrisk 2^{-k}

- ▶ Om $AB = C$ svarar algoritmen alltid "sant".
- ▶ Om $AB \neq C$ blir den lurad i ett varv med sannolikhet högst $1/2$ — och i alla k oberoende varv med sannolikhet högst 2^{-k} .
- ▶ Felet är *ensidigt*: "falskt" är alltid rätt.

k	felrisk högst
1	$1/2$
10	$\approx 10^{-3}$
20	$\approx 10^{-6}$
30	$\approx 10^{-9}$

Ett "falskt" kommer med ett bevis: vektorn r som visar att $Cr \neq A(Br)$. Det är bara "sant" som kan vara fel.

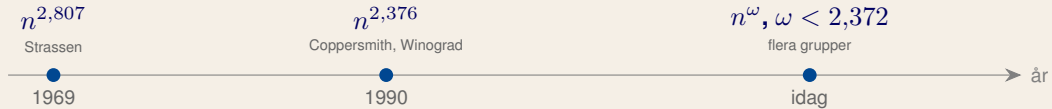
Kaninhål: Större slumpstal, färre varv

Slumpa $r \in \{1, 2, \dots, s\}^n$ i stället för $\{0, 1\}^n$.

- ▶ Håll alla koordinater utom r_k fasta. Då är $(Dr)_i = c + d_{ik}r_k$ för något tal c .
- ▶ Det är noll för högst *ett* värde på r_k . Alltså lurad med sannolikhet högst $1/s$.
- ▶ Med $s = 2^k$ räcker *ett* varv för felrisk 2^{-k} — men talen blir k bitar större.

Samma resonemang för polynom i många variabler heter Schwartz–Zippels lemma: ett nollskilt polynom av grad d är noll i en slumpad punkt med sannolikhet högst d/s . Freivalds algoritm (1977) är fallet $d = 1$.

Kaninhål: Att kontrollera är lättare än att räkna ut



Freivalds (1977): kontroll i $O(n^2)$. Lägre går inte, för alla n^2 tal måste läsas.

Ingen vet om exponenten ω för matrismultiplikation är 2. Viggos 2,376 är från 1990, och sedan dess har rekordet bara krupit ned några tusendelar. Kontrollen har legat på n^2 hela tiden.

Vanliga missuppfattningar om Freivalds algoritm

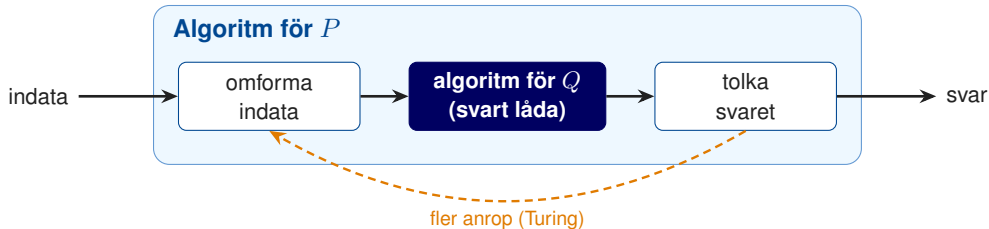
⚠ Missuppfattning: " $(AB)r = A(Br)$, så det spelar ingen roll." Värdet är detsamma, priset är det inte. Hela poängen är att aldrig räkna ut AB .

⚠ Missuppfattning: "Felrisk 1/2: hälften av svaren är fel." Nej. Bara "sant" kan vara fel, bara när $AB \neq C$, och högst med sannolikhet 1/2 per varv.

⚠ Missuppfattning: "Fler fel i C syns lättare." Inte alltid: med raden $(1, -1)$ i D blir $(Dr)_i = r_1 - r_2$, som är noll för hälften av alla r . Två fel, samma risk.

⚠ Missuppfattning: "Slumpa r en gång och testa k gånger." Samma r ger samma svar. Varje varv behöver ett nytt, oberoende r .

Reduktion: lös P med hjälp av Q

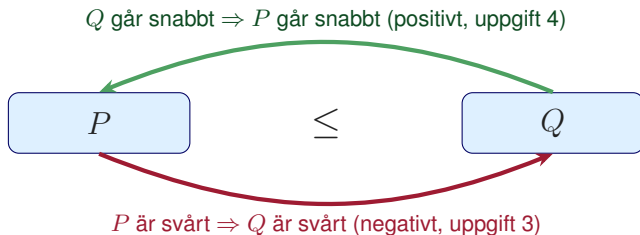


Karpreduktion: ett enda anrop, och svaret returneras direkt. Ingen tolkning.

Turingreduktion: anropa Q flera gånger — polynomiskt många — och kombinera svaren.

Vi skriver $P \leq Q$: " P reduceras till Q ". Kostnaden är omformningen plus anropen.

Åt vilket håll? Det är hela poängen



$P \leq Q$: Q är *minst lika* svårt som P , upp till reduktionens kostnad. Algoritmer går bakåt längs pilen, svårighet går framåt.

⚠ Missuppfattning: "Mitt problem kan reduceras till ett svårt problem, alltså är det svårt." Fel håll. Att avgöra om ett tal är jämnt går att reducera till graffärgning. Det gör inte jämnhet svårt.

Tre sorters problem: BOK och BIO

BOK

Beslutsproblem

värdes**O**ptimeringsproblem

Konstruktionsoptimeringsproblem

"Kan G färgas med k färger?"

"Hur få färger behövs?"

"Färga G med så få färger som möjligt."

BIO

Bool

Integer

Objekt

(Minnesregeln är min, från när det hette optimeringsproblem. Nu har O:et flyttat in i ordet. Regeln överlever, med nöd.)

Nedåt är gratis. Har man en optimal färgning vet man minsta antalet färger, och då kan man svara på beslutsfrågan.

Uppåt kostar. Från beslut till värde: binärsökning (uppgift 5 och 6a). Från värde till konstruktion: fråga om ändrade indata (uppgift 6b).

3

Reduktion som ger negativt resultat

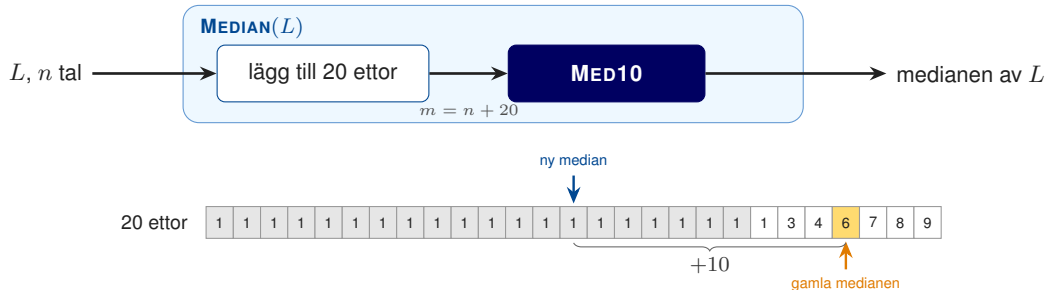
Målet: använda en känd undre gräns för medianen till att få en undre gräns för ett närliggande problem.

Att se upp med:

- ▶ Riktningen: reducera det *kända* svåra problemet *till* det nya.
- ▶ Hur många tal som ska läggas till — och varför det inte är 10.
- ▶ Att uttrycka gränsen i det nya problemets storlek.

Uppgift 3: Reduktion som ger negativt resultat

Att hitta medianelementet (det $\lceil n/2 \rceil$:e minsta elementet) av n positiva heltal kräver minst $2n$ jämförelser i värsta fallet. Utnyttja detta resultat för att bestämma en undre gräns för antalet jämförelser som krävs för att hitta det $(\lceil n/2 \rceil + 10)$:e minsta elementet.



$L = [3, 9, 4, 8, 1, 6, 7]$ har medianen 6. Med 20 ettor är $m = 27$ och $\lceil 27/2 \rceil + 10 = 24$ — plats 24 är 6.

Räkningen

- ▶ $f(n)$ = jämförelser som MEDIAN gör på n tal.
- ▶ $g(m)$ = jämförelser som MED10 gör på m tal.
- ▶ MEDIAN gör inget annat än att anropa MED10:
 $f(n) = g(n + 20)$.
- ▶ Känt: $f(n) \geq 2n$. Alltså $g(n + 20) \geq 2n$.
- ▶ Sätt $m = n + 20$: $g(m) \geq 2(m - 20) = \mathbf{2m - 40}$.

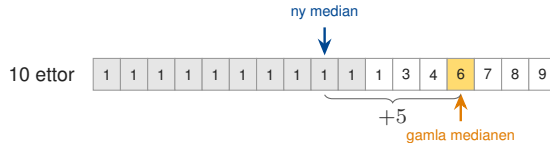
(Uppställningen med f och q : Anton Grensjö, Lisa Li)

Varför ettor? 1 är det minsta positiva heltalet, så ettorna hamnar alltid först. Finns det redan ettor i L gör det inget: svaret är ett *värde*, och värdet blir rätt.

Gränsen gäller för $m \geq 21$, där det finns utfyllda indata. För $m \leq 20$ lovar den ingenting, vilket är ärligt.

Svar: att hitta det $(\lceil m/2 \rceil + 10)$:e minsta av m tal kräver minst $2m - 40$ jämförelser i värsta fallet — oavsett algoritm.

Varför 20 ettor och inte 10?



Tio ettor flyttar gamla medianen tio steg uppåt. Men medianen själv flyttar sig också, fem steg. Varje tillagt litet tal flyttar skillnaden ett halvt steg: 20 ettor ger +10.

⚠ Missuppfattning: "Lägg till 10 ettor." Då hamnar gamla medianen på plats $\lceil m/2 \rceil + 5$.

⚠ Missuppfattning: "Lägg till 20 stora tal." Då står gamla medianen kvar medan medianen flyttar sig uppåt: $\lceil m/2 \rceil - 10$.

Fel håll ger fel sorts resultat

MEDIAN

$\leq$

MED10

utfyllnad med 20 ettor
 $\Rightarrow$ **undre** gräns för MED10

MED10

$\leq$

MEDIAN

utfyllnad med 20 stora tal
 $\Rightarrow$ **övre** gräns för MED10

Båda reduktionerna är korrekta. Bara den första svarar på frågan: för en undre gräns ska det *kända* svåra problemet stå till vänster.

⚠ Missuppfattning: "Undre gränsen är $2n$." Den gäller medianproblemets n . Uttryckt i det nya problemets storlek blir den $2m - 40$ — i uppgiften heter båda n , men de är inte samma tal.

Kaninhål: Hur svårt är medianen egentligen?

- ▶ **Undre gräns, deterministiskt:** ungefär $2n$ (Bent och John, 1985). Dor, Håstad, Ulfberg och Zwick visade $2,01n$ för algoritmer som börjar med att jämföra tal parvis — Håstad och Ulfberg från KTH. Dor och Zwick visade sedan $(2 + \varepsilon)n$ för alla algoritmer, med $\varepsilon \approx 2^{-80}$.
- ▶ **Bästa deterministiska algoritmen:** ungefär $2,95n$ (Dor och Zwick). Glappet mellan 2 och 2,95 har stått still i årtionden.
- ▶ **Med slump:** $1,5n + o(n)$ jämförelser i förväntan (Floyd och Rivest), och bättre går inte (Cunto och Munro).

Slump hjälper alltså — bevisligen. Ingen deterministisk algoritmen klarar $2n$, men en slumpad klarar $1,5n$ i förväntan. Dagens två teman möts här.

Kaninhål: Samma knep, andra platser

- ▶ Det $(\lceil m/2 \rceil + c)$:e minsta, $c \geq 0$: lägg till $2c$ ettor. Undre gräns $2(m - 2c) = 2m - 4c$.
- ▶ Det $(\lceil m/2 \rceil - c)$:e minsta: lägg till $2c$ stora tal i stället.
- ▶ Långt från mitten blir det lättare. Minimum kräver bara $n - 1$ jämförelser: varje tal utom minimum måste förlora minst en jämförelse, och varje jämförelse har bara en förlorare.

Utfyllnad är reduktionernas vardagsverktyg: ändra indata så att det kända problemets svar blir det nya problemets svar. I uppgift 6 ändrar vi en graf i stället för en lista.

4

Reduktion som ger positivt resultat

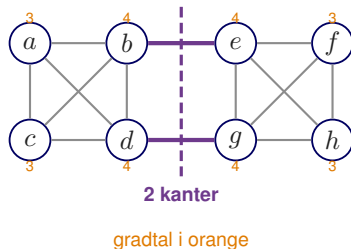
Målet: avgöra om K borttagna kanter räcker för att dela grafen — genom att reducera till maxflöde.

Att se upp med:

- ▶ Minsta antalet kanter är storleken på ett minsta snitt, $\lambda(G)$.
- ▶ Maxflöde ger snittet mellan *två givna* hörn. Vilka hörn?
- ▶ $\lambda(G)$ är inte minsta gradtalet.

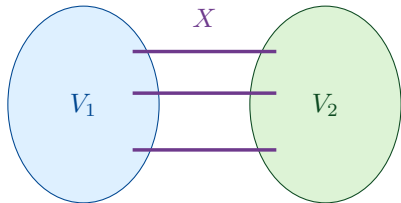
Uppgift 4: Reduktion som ger positivt resultat

Kantsammanhangsproblemet. Indata: en sammanhängande oriktad graf $G = (V, E)$ och ett positivt heltal K mellan 1 och $|V|$. Fråga: räcker det att ta bort K kanter för att göra G osammanhängande? Lös problemet genom att reducera det till ett problem vi redan kan lösa.



- Låt $\lambda(G)$ vara minsta antalet kanter vars borttagning delar G .
- Svaret är ja precis när $K \geq \lambda(G)$.
- Här: minsta gradtalet är 3, men två kanter räcker. $\lambda(G) = 2$.

En minimal kantmängd är ett snitt

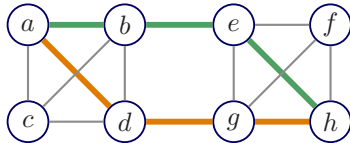


Låt X vara en *minimal* kantmängd vars borttagning gör G osammanhängande (ingen äkta delmängd räcker).

- ▶ Då delas G i *exakt två* komponenter, V_1 och V_2 — annars kunde en kant läggas tillbaka.
- ▶ Alla kanter i X går mellan V_1 och V_2 — av samma skäl.
- ▶ X är alltså ett snitt, och $\lambda(G)$ är storleken på ett minsta snitt.

(Resonemanget: Viggo Kann; bilden med s och t längre fram: Alice)

Snitt = flöde = kantdisjunkta stigar

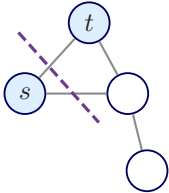


Två kantdisjunkta stigar från a till h .

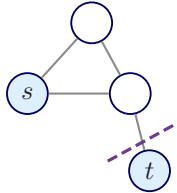
- ▶ Ge varje kant kapacitet 1 i båda riktningarna: $\{u, v\}$ blir $u \rightarrow v$ och $v \rightarrow u$.
- ▶ Maxflöde från s till t = minsta antalet kanter som skiljer s från t (maxflöde–minsnitt).
- ▶ Det är också största antalet kantdisjunkta s – t -stigar (Mengers sats).

⚠ Missuppfattning: "Rikta varje kant åt ett håll." Då kan flödet bara gå åt det hållet, och maxflödet kan bli för litet. Kanten måste kunna användas åt båda hållen.

Vilka s och t ? Fixera s , variera t



Maxflöde 2.



Maxflöde 1.

Ett minsta snitt (V_1, V_2) delar V . Hörnet s ligger på någon sida, och något t ligger på den andra. För det t är maxflödet $\lambda(G)$. För alla andra t är det minst $\lambda(G)$.

$$\lambda(G) = \min_{t \in V - \{s\}} \text{MAXFLOW}(G', s, t)$$

$|V| - 1$ flödesberäkningar räcker. Alla par (s, t) behövs inte — s hamnar ju alltid på någon sida av snittet. (Exemplet: Alice)

Algoritmen

Kantsam(G, K)

- 1: välj ett godtyckligt hörn s
- 2: $G' \leftarrow G$ med kapacitet 1 åt båda håll
- 3: $\lambda \leftarrow \min_{t \in V - \{s\}} \text{MAXFLOW}(G', s, t)$
- 4: **returnera** $K \geq \lambda$

(Efter Lisa Li)

- ▶ $|V| - 1$ anrop till MAXFLOW.
- ▶ Varje anrop i $O(|V|^3)$ med en bra flödesalgoritm.
- ▶ Totalt $O(|V|^4)$.

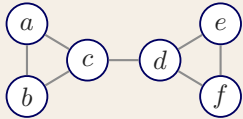
Om $K > |E|$ finns det inte K kanter att ta bort. Det kan bara hända när G är ett träd och $K = |V|$. Läs "räcker det att ta bort K kanter" som "högst K ", så är svaret ja.

Kaninhål: Att avgöra är billigare än att räkna ut

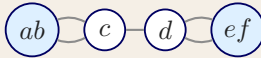
- ▶ Alla kapaciteter är 1. Ford–Fulkerson ökar flödet med 1 per varv, och varje varv är en grafsökning, $O(|E|)$.
- ▶ Flödet ut från s är högst $\deg(s)$. Välj s med minsta grad.
- ▶ Beslutsfrågan behöver inte λ exakt. Avbryt varje flödesberäkning när flödet når $K + 1$: då vet vi att just det t inte räcker.
- ▶ Tid: $O(|V| \cdot K \cdot |E|)$ i stället för $O(|V|^4)$.

Med $K = 1$ frågar man efter en *brygga*: en kant vars borttagning delar grafen. Den hittas med en enda djupetförstsökning i tid $O(|V| + |E|)$.

Kaninhål: Samma problem som Monte Carlo: Kargers algorit



Två trianglar och en brygga.



Efter två slumpade kontraktioner. Parallella kanter behålls.



Två hörn kvar. Kanten emellan är ett snitt.

Algoritm: slå ihop ändpunkterna på en slumpvis vald kant tills två hörn återstår.

- ▶ Minsta snittet har c kanter $\Rightarrow$ varje hörn har $\text{grad} \geq c \Rightarrow |E| \geq cn/2$. Risken att välja en snittkant är högst $2/n$.
- ▶ Lyckas med sannolikhet minst $\prod_{i=3}^n (1 - \frac{2}{i}) = \frac{2}{n(n-1)}$. Upprepa $\frac{n^2}{2} \ln n$ gånger: felrisk högst $1/n$.

Vanliga missuppfattningar om kantsammanhang

⚠ Missuppfattning: " $\lambda(G)$ är minsta gradtalet."

Det är en övre gräns: ta bort alla kanter vid ett hörn. I hanteln är minsta gradtalet 3 och $\lambda(G) = 2$.

⚠ Missuppfattning: "Maxflödet mellan två hörn ger svaret." Det ger minsta snittet *mellan just dem*, som kan vara större: i s/t -exemplet 2 i stället för 1.

⚠ Missuppfattning: "Man måste prova alla par (s, t) ." Det är $O(|V|^2)$ flödesberäkningar. Ett fixt s och $|V| - 1$ val av t räcker.

⚠ Missuppfattning: "Ta bort kanter vid hörnet med lägst grad." Ger en övre gräns, inte svaret. Minsta snittet kan ligga mitt i grafen.

5

Värdesoptimeringsproblem som beslutsproblem

Målet: formulera ett beslutsproblem som räcker för att hitta optimum — och hitta optimum med polynomiskt många frågor.

Att se upp med:

- ▶ Frågan måste vara monoton: "högst I ", inte "exakt I ".
- ▶ Binärsökningen går över *värdet*, inte över uppgifterna.
- ▶ Polynomiskt i indatas längd: $\log W$ frågor, inte W .

Uppgift 5: Jobbmaskning

Indata: arbetsdagens längd T och n arbetsuppgifter. Uppgift i tar tiden t_i och kräver arbetsinsatsen w_i . Alla tal är positiva heltal. **Lösning:** ett urval $D \subseteq [1..n]$ som fyller ut dagen: $\sum_{i \in D} t_i \geq T$. **Mål:** minimera arbetsinsatsen $\sum_{i \in D} w_i$.

Uppgift ($T = 13$)	t_i	w_i
Läsa mejl	3	3
Fika med chefen	5	1
Rätta labbar	3	4
Skriva rapport	6	5
Planeringsmöte	6	2

Optimum: mejl, fika och planeringsmöte. 14 timmar, insats 6.

Girigt efter t_i/w_i : fika, planeringsmöte, rapport. 17 timmar, insats 8.

Problemet är en variant av kappsäcksproblemet och NP-svårt. Girigt hjälper inte.

a) Beslutsproblemet

Indata: T , ett målvärde I och n uppgifter med t_i och w_i .

Fråga: finns det $D \subseteq [1..n]$ med $\sum_{i \in D} t_i \geq T$ och $\sum_{i \in D} w_i \leq I$?

Svaret är *monotont* i I : är det ja för I så är det ja för alla större I . Exemplet:

I :	nej	nej	nej	nej	nej	nej	ja	ja	ja	ja	ja	ja	ja	ja	ja	ja
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Det är monotonin som gör binärsökning möjlig. Optimum är där svaret slår om: 6.

b) Turingreduktion: binärsök efter minsta I

MinInsats($T, \{t_i\}, \{w_i\}$)

```

1:  $W \leftarrow \sum_i w_i$ 
2: om inte BESLUT( $T, W, \dots$ ) så
3:   returnera ingen lösning
4:  $lo \leftarrow 0; hi \leftarrow W$ 
5: medan  $lo < hi$  gör
6:    $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
7:   om BESLUT( $T, mid, \dots$ ) så
8:      $hi \leftarrow mid$ 
9:   annars
10:     $lo \leftarrow mid + 1$ 
11: returnera  $lo$ 

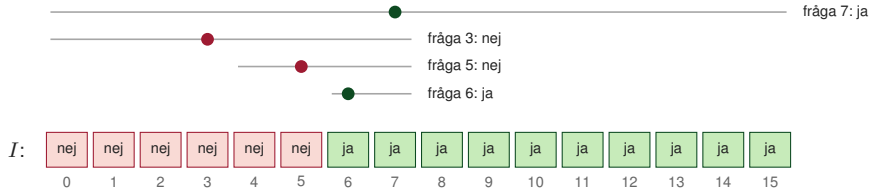
```

$$\triangleright \sum t_i < T$$

- ▶ Optimum ligger mellan 0 och $W = \sum_i w_i$.
- ▶ Invariant: svaret är ja för hi och nej för allt under lo .
- ▶ $\lceil \log_2(W + 1) \rceil + 1$ anrop till beslutsalgoritmen.

⚠ Missuppfattning: "Binärsök bland uppgifterna" eller "bland delmängderna". Sökningen går över *målvärdet* I . Uppgifterna sköter beslutsalgoritmen.

Binärsökningen på exemplet



$W = 15$. Först kontrollen $I = 15$: ja, det finns en lösning. Sedan frågor om 7, 3, 5 och 6 — fyra frågor, och svaret är 6. Varje fråga halverar intervallet, så $W = 10^9$ hade krävt 30.

Polynomiskt i vad?

- ▶ Indata är $2n + 1$ tal, skrivna med bitar. Talet $W = \sum w_i$ tar bara $O(\log W)$ bitar.
- ▶ **Binärsökning:** $O(\log W)$ anrop — högst ungefär lika många som indata har bitar. Polynomiskt.
- ▶ **Linjär sökning** $I = 0, 1, 2, \dots$: upp till W anrop. Exponentiellt i antalet bitar.

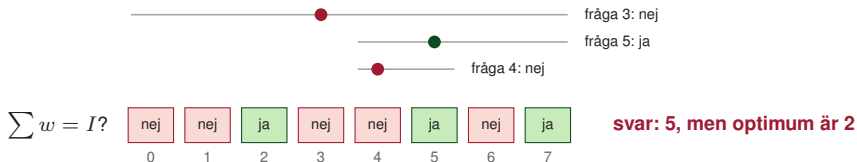
W	binärt	linjärt
15	5	16
10^6	21	10^6
10^9	31	10^9

Antal anrop till beslutsalgoritmen.

Polynomiskt betyder polynomiskt i indatas *längd*. Ett tal W är bara $\log W$ bitar långt, så en algoritm som är linjär i W är pseudopolynomisk.

Varför ”högst I ” och inte ”exakt I ”?

Liten instans: dagen är 1 timme. Uppgift A: 1 timme, insats 2. Uppgift B: 1 timme, insats 5. Möjliga insatser: 2, 5 och 7. Frågan ”finns D med $\sum w_i = I$?”:



Svaret slår om fram och tillbaka. Binärsökningen hamnar på 5 — fast optimum är 2.

⚠ Missuppfattning: ”Finns det ett dagsverke med insats *exakt* I ?” Den frågan är inte monoton, och binärsökningen ger fel svar utan att märka det.

Kaninhål: Och själva dagsverket?

Optimum är 6. Gå igenom uppgifterna en i taget och fråga: går insats 6 fortfarande *utan* den här?

- ▶ Ja: stryk uppgiften för gott.
- ▶ Nej: den behövs. Behåll den.

Det som blir kvar är ett optimalt dagsverke. Totalt $O(\log W + n)$ anrop.

Utan ...	insats 6?	
läsa mejl	nej	behåll
fika med chefen	nej	behåll
rätta labbar	ja	stryk
skriva rapport	ja	stryk
planeringsmöte	nej	behåll

Samma knep som i uppgift 6b: ändra indata och fråga igen. Ett beslutsorakel räcker för att *konstruera* lösningen, inte bara för att hitta dess värde.

Vanliga missuppfattningar om uppgift 5

⚠ Missuppfattning: "Ta uppgifterna med bäst t_i/w_i först." Ger insats 8 i exemplet, där optimum är 6. Problemet är NP-svårt.

⚠ Missuppfattning: "Prova $I = 0, 1, 2, \dots$ tills svaret blir ja." Rätt svar, men upp till W anrop: exponentiellt i indatas längd.

⚠ Missuppfattning: "Det är en Karpreduktion." Beslutsalgoritmen anropas många gånger och svaren kombineras: en Turingreduktion.

⚠ Missuppfattning: "Det finns alltid en lösning." Inte om $\sum_i t_i < T$. Då svarar beslutsalgoritmen nej även för $I = W$ — därför den första kontrollen.

6

Besluts-, värdes- och konstruktionsproblem

Målet: gå från ett ja/nej-orakel för graffärgning till minsta antalet färger, och vidare till en färgning — med polynomiskt många frågor.

Att se upp med:

- ▶ Oraklet svarar bara ja eller nej om *hela* grafen.
- ▶ Beslut kodas in i grafen: en ny kant betyder ”olika färger”.
- ▶ Polynomiskt många anrop är inte samma sak som polynomisk tid.

Uppgift 6 och del a

Anta att algoritmen $\text{GRAPHCOLOURING}(G, k)$ på tid $T(n)$ svarar 1 om hörnen i G kan färgas med k färger utan att någon kant har likfärgade ändpunkter. **a)** Bestäm det minimala antalet färger i tid $O(\log n \cdot T(n))$. **b)** Färga hörnen med minimalt antal färger i tid $O(P(n)T(n))$, där $P(n)$ är ett polynom.

MinFärger(G)

```

1:  $lo \leftarrow 1; hi \leftarrow n$ 
2: medan  $lo < hi$  gör
3:    $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
4:   om  $\text{GRAPHCOLOURING}(G, mid)$  så
5:      $hi \leftarrow mid$ 
6:   annars
7:      $lo \leftarrow mid + 1$ 
8: returnera  $lo$ 

```

- ▶ Svaret ligger mellan 1 och n , och det är monotont: k -färgbar $\Rightarrow (k + 1)$ -färgbar.
- ▶ $\lceil \log_2 n \rceil$ anrop: $O(\log n \cdot T(n))$.
- ▶ $k = 1$ och $k = 2$ är lätta (inga kanter; bipartit, BFS). Oraklet behövs från $k = 3$, där problemet är NP-fullständigt.

b) Idén: fråga genom att ändra grafen

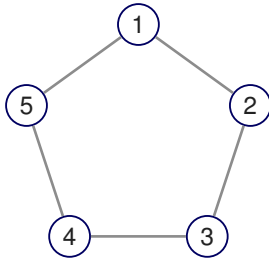
- ▶ Oraklet kan inte svara på "vilken färg ska v ha?".
- ▶ Men en ny kant (u, v) *tvingar* u och v att få olika färger.
- ▶ Är grafen med kanten fortfarande k -färgbar? Behåll kanten.
- ▶ Annars *måste* v ha samma färg som u .

(Algoritmen: Viggo Kann)

CreateColouring($G = (V, E), k$)

- 1: $u \leftarrow$ första hörnet i V
- 2: $C \leftarrow \{u\}; u.färg \leftarrow k$
- 3: **för alla** $v \in V - \{u\}$ **gör**
- 4: **om** $(u, v) \notin E$ **så**
- 5: **om** GRAPHCOLOURING($(V, E \cup \{(u, v)\}), k$)
- 6: **så**
- 7: $E \leftarrow E \cup \{(u, v)\}$
- 8: **annars**
- 9: $C \leftarrow C \cup \{v\}; v.färg \leftarrow k$
- 10: **om** $k > 1$ **så**
- 11: CREATECOLOURING($(V - C, E), k - 1$)

CreateColouring på C_5 (Alices exempel)

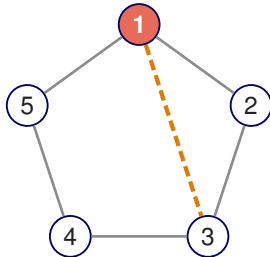


k u prova kant oraklet

orange: tillagda kanter streckad röd: oraklet sa nej

C_5 behöver tre färger ($\chi = 3$, udda cykel). Börja med $k = 3$ och $u = 1$, som får färg 3.

CreateColouring på C_5 (Alices exempel)

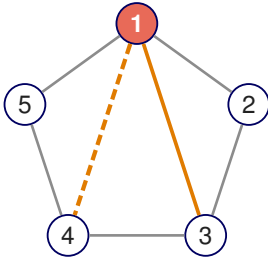


k u prova kant oraklet

orange: tillagda kanter streckad röd: oraklet sa nej

Måste 3 ha samma färg som 1? Lägg till kanten 1–3 och fråga oraklet med $k = 3$.

CreateColouring på C_5 (Alices exempel)

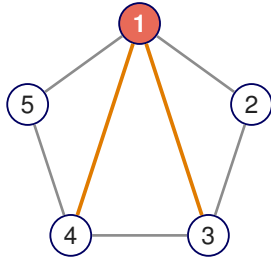


k	u	prova kant	oraklet
3	1	1–3	ja: behåll
3	1	1–4	ja: behåll

orange: tillagda kanter streckad rød: oraklet sa nej

Svar: ja, fortfarande 3-färgbar. Kanten får vara kvar. Nu 1–4: också ja.

CreateColouring på C_5 (Alices exempel)

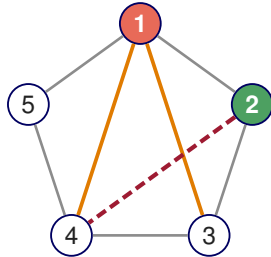


k	u	prova kant	oraklet
3	1	1-3	ja: behåll
3	1	1-4	ja: behåll

orange: tillagda kanter streckad röd: oraklet sa nej

2 och 5 är redan grannar till 1. Färgklassen är $\{1\}$. Fortsätt utan 1, med $k = 2$.

CreateColouring på C_5 (Alices exempel)

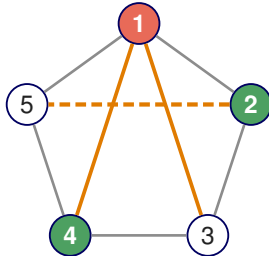


k	u	prova kant	oraklet
3	1	1–3	ja: behåll
3	1	1–4	ja: behåll
2	2	2–4	nej: 4 får 2:s färg

orange: tillagda kanter streckad röd: oraklet sa nej

$u = 2$ får färg 2. Lägg till 2–4: triangeln 2–3–4 går inte med två färger. Svar: nej.

CreateColouring på C_5 (Alices exempel)

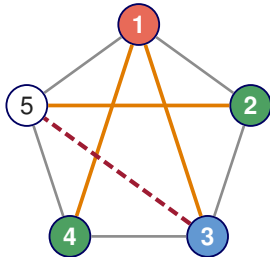


k	u	prova kant	oraklet
3	1	1–3	ja: behåll
3	1	1–4	ja: behåll
2	2	2–4	nej: 4 får 2:s färg
2	2	2–5	ja: behåll

orange: tillagda kanter streckad röd: oraklet sa nej

Alltså måste 4 ha samma färg som 2. Sedan 2–5: svar ja, kanten får vara kvar.

CreateColouring på C_5 (Alices exempel)

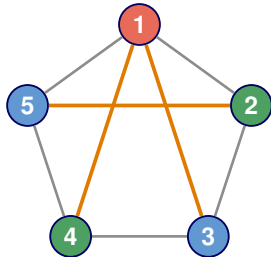


k	u	prova kant	oraklet
3	1	1–3	ja: behåll
3	1	1–4	ja: behåll
2	2	2–4	nej: 4 får 2:s färg
2	2	2–5	ja: behåll
1	3	3–5	nej: 5 får 3:s färg

orange: tillagda kanter streckad röd: oraklet sa nej

Kvar: $\{3, 5\}$ med $k = 1$. $u = 3$ får färg 1. Kanten 3–5 går inte med en färg: nej.

CreateColouring på C_5 (Alices exempel)



k	u	prova kant	oraklet
3	1	1–3	ja: behåll
3	1	1–4	ja: behåll
2	2	2–4	nej: 4 får 2:s färg
2	2	2–5	ja: behåll
1	3	3–5	nej: 5 får 3:s färg

orange: tillagda kanter streckad röd: oraklet sa nej

Klart: tre färger och fem frågor till oraklet. Färgningen är giltig — kontrollera själva.

Varför blir det rätt — och hur dyrt är det?

- ▶ Grafen är hela tiden k -färgbar: vi behåller bara kanter som oraklet godkänt.
- ▶ När slingan är klar har varje $v \notin C$ en kant till u , och varje $v \in C$ har samma färg som u i *varje* k -färgning. C är alltså precis u :s färgklass.
- ▶ Utan C räcker $k - 1$ färger. Rekursionen färgar resten.

Tid. Oraklet anropas högst en gång per hörnpar: $O(n^2)$ anrop. Med del a:

$$O(\log n \cdot T(n) + n^2 \cdot T(n)) \\ = O(n^2 \cdot T(n)).$$

⚠ Missuppfattning: "Ta bort C och fortsätt med samma k ." Då får nästa färgklass också färg k , och två grannar i olika klasser kan bli likfärgade.

Kaninhål: En annan väg: slå ihop i stället för att färga

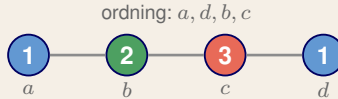
För varje par u, v som inte är grannar:

- ▶ Är $G + (u, v)$ fortfarande k -färgbar? Lägg till kanten.
- ▶ Annars: slå ihop u och v till ett hörn. De måste ju ha samma färg.

När inga par finns kvar är grafen fullständig med k hörn. Varje hopslaget hörn är en färgklass. Också $O(n^2)$ anrop.

Båda vägarna är *självreduktion*: konstruktionsproblemet löses med ett orakel för beslutsproblemet, genom att ändra indata och fråga igen. Det går för alla NP-fullständiga problem, och det är därför man i komplexitetsteorin nöjer sig med att studera beslutsproblem.

Kaninhål: Girig färgning räcker inte



Girigt: ta hörnen i en viss ordning och ge varje hörn den minsta färg som ingen granne har. Stigen behöver två färger, men i ordningen a, d, b, c blir det tre.

Med rätt ordning blir girig färgning optimal — det finns alltid en sådan ordning. Att hitta den är lika svårt som att färga. Det finns grafer som behöver två färger där girig färgning i sämsta ordning använder $n/2$.

Vanliga missuppfattningar om uppgift 6

⚠ Missuppfattning: "Fråga oraklet vilken färg v ska ha." Det svarar bara ja eller nej om hela grafen. Delbeslut måste kodas in i grafen, som kanter eller sammanslagningar.

⚠ Missuppfattning: "Binärsökningen fungerar för att färgerna är sorterade." Den fungerar för att svaret är monotont i k .

⚠ Missuppfattning: " $O(n^2 \cdot T(n))$ är polynomisk tid." Bara om $T(n)$ är polynomisk, och för $k \geq 3$ vet ingen hur man får det. Reduktionen visar att konstruera är *högst polynomiskt svårare* än att avgöra.

⚠ Missuppfattning: "Det räcker att fråga en gång per hörn." Algoritmen frågar en gång per hörnpar upp till n^2 gånger.

Innehåll, som avslutning

Uppgift	Det man ska ha med sig
1 Skipplistor	$p = 1/e$ är optimalt i ett mått med platt minimum. $p = 2^{-k}$ ger billiga slumpbitar, $p = 1/4$ dessutom färre pekare.
2 Matrisverifikation	$A(Br)$, inte $(AB)r$. Ensidigt fel, minst hälften av alla r avslöjar, k varv ger 2^{-k} .
3 Negativt resultat	Reducera det kända svåra problemet <i>till</i> det nya. 20 ettor, inte 10: $2m - 40$.
4 Positivt resultat	$\lambda(G)$ = minsta snittet = $\min_t$ maxflöde från ett fixt s . Inte minsta gradtalet.
5 Optimering som beslut	"Högst I ", monotont. Binärsökning över värdet: $O(\log W)$ anrop, polynomiskt.
6 Graffärgning	Koda delbeslut i grafen och fråga igen. $O(n^2)$ anrop till oraklet.

Två teman, en gemensam fråga: *vad kostar svaret, och i vilket mått?* Slumpbitar och felrisk i första halvan, anrop och indatas längd i andra.



Frågor?

(Svaren är rätt med sannolikhet minst $1 - 2^{-k}$.)